Лабораторная работа №8

Реализация основных моделей в дискретно-событийном подходе.

Талебу Тенке Франк

Содержание

1 Цель работы 1
2 Задание 1
3 Теоретическое введение 2
3.1 Программный комплекс 2
3.2 Входные и выходные данные 2
4 Выполнение лабораторной работы 2
4.1 Программный комплекс 2
4.2 Дополнительные задания 5
5 Выводы 9
Список литературы 9

1 Цель работы

- Изучить дискретно-событийный подход к имитационному моделированию на примере классической модели распространения инфекции SIR. Реализовать стохастическую дискретно-событийную модель в виде программного комплекса на языке Julia. Провести анализ влияния параметров, сравнить со стохастической и детерминированной версиями, оценить производительность и модифицировать модель.

2 Задание

1. Программный комплекс
- Структура программы (sir_model.jl)
- Скрипт запуска (sir_des.jl)
2. Дополнительные задания
- Анализ чувствительности к параметрам
- Детерминированная длительность болезни
- Оценка производительности
- Сохранение результатов в CSV
- Добавление демографических событий
- Вакцинация

- Модель SEIR

3 Теоретическое введение

3.1 Программный комплекс

Программный комплекс состоит из двух основных файлов: - src/sir_model.jl (ядро модели): определение структур данных, функций событий, логики агентов и управления симуляцией.

- scripts/sir_des.jl (скрипт запуска): задание параметров, инициализация модели, выполнение прогона и визуализация.

3.2 Входные и выходные данные

Входные данные

1. Начальное состояние популяции u0 (вектор из трёх целых чисел): число восприимчивых, инфицированных и переболевших на момент времени 0.

2. Параметры модели p (вектор из трёх вещественных чисел):

- $/beta$ (вероятность передачи при контакте) – число от 0 до 1.
- c (частота контактов) – среднее число контактов индивида в единицу модельного времени.
- $/gamma$ (интенсивность выздоровления) – величина > 0; обратная к среднему времени болезни.

3. Длительность симуляции tmax (вещественное число) – момент остановки модельного времени.

4. Зерно генератора случайных чисел – опционально, для воспроизводимости.

Выходные данные

1. Таблица DataFrame с колонками:
- t – моменты времени наступления событий.
- S – число восприимчивых.
- I – число инфицированных.
- R – число переболевших.
2. График временных рядов S, I, R, сохранённый в plots/sir_des.png.

4 Выполнение лабораторной работы

4.1 Программный комплекс

Структура программы (sir_model.jl)

- Здесь я создала (“src/sir_model.jl”), я написала код и запустила его.(рис. 1).

Ouvrir

sir_model.jl
~/work/study/2025-2026/simulation-modeling/study_2025-2026

```julia
using ResumableFunctions, ConcurrentSim, Distributions, DataFrames, Random

# Вспомогательные функции для обновления массивов состояния
function increment!(a::Array{Int64})
    push!(a, a[length(a)] + 1)
end
function decrement!(a::Array{Int64})
    push!(a, a[length(a)] - 1)
end
function carryover!(a::Array{Int64})
    push!(a, a[length(a)])
end
# Структуры данных
mutable struct SIRPerson
    id::Int64
    status::Symbol # :S, :I, :R
end
mutable struct SIRModel
    sim::ConcurrentSim.Simulation # Тип Simulation, не Environment
    β::Float64
    c::Float64
    γ::Float64
    ta::Array{Float64}
    Sa::Array{Int64}
    Ia::Array{Int64}
    Ra::Array{Int64}
    allIndividuals::Array{SIRPerson}
end

# Функции обновления статистики при событиях
function infection_update!(sim::ConcurrentSim.Simulation, m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
    decrement!(m.Sa)
    increment!(m.Ia)
    carryover!(m.Ra)
end
function recovery_update!(sim::ConcurrentSim.Simulation, m::SIRModel)
    push!(m.ta, ConcurrentSim.now(sim))
```

Рис. 1: Рисунок 1

- Отображение на консоли. (рис. 2).

```
julia> include("src/sir_model.jl")
out (generic function with 1 method)
```

Рис. 2: Рисунок 2

Скрипт запуска (sir_des.jl)

- Здесь я создала (“scripts/sir_des.jl”), я написала код и запустила его.(рис. 3).

sir_des.jl

```julia
using DrWatson
@quickactivate "project"
include(srcdir("sir_model.jl"))
using Random, StatsPlots, BenchmarkTools

# Параметры модели
tmax = 40.0
u0 = [990, 10, 0] # S, I, R
p = [0.05, 10.0, 0.25] # β, c, γ

Random.seed!(1234)

# Запуск модели
des_model = MakeSIRModel(u0, p)
activate(des_model)
sir_run(des_model, tmax)
data_des = out(des_model)

# Визуализация
@df data_des plot(
    :t,
    [:S :I :R],
    labels = ["S" "I" "R"],
    xlab = "Время",
    ylab = "Численность",
    title = "Дискретно-событийная SIR модель",
)
savefig(plotsdir("sir_des.png"))
```

Рис. 3: Рисунок 3

- График, который мы получили. (рис. 4).

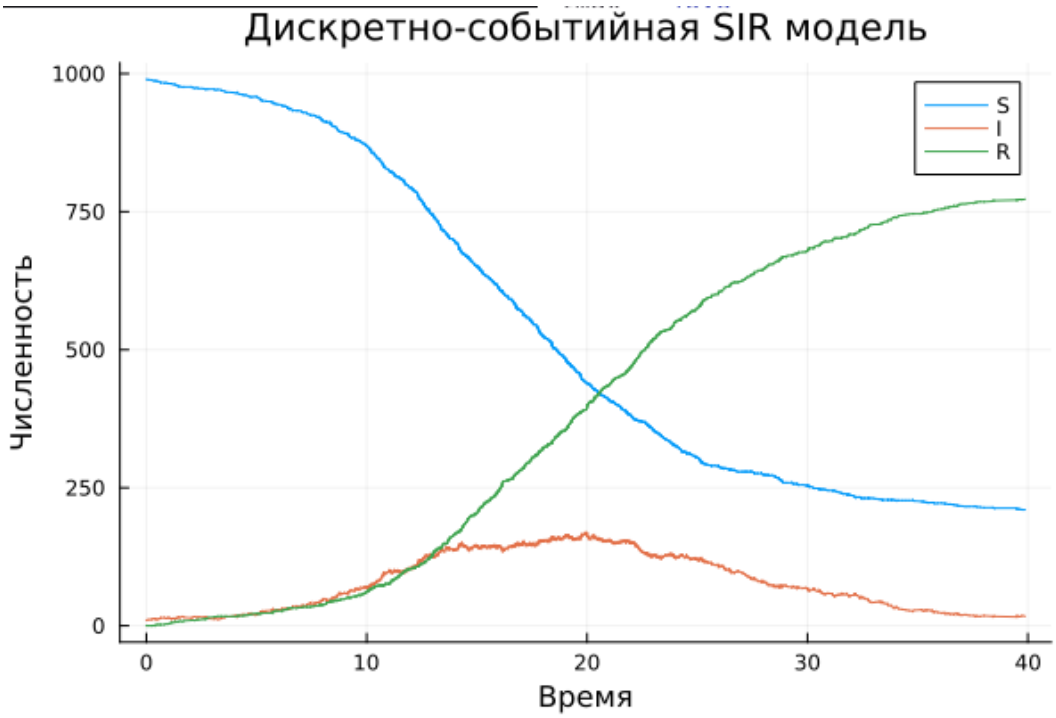


Рис. 4: Рисунок 4

4.2 Дополнительные задания

Анализ чувствительности к параметрам

- Написать цикл, варьирующий параметры и сохраняющий результаты. (рис. 5).

```
# настройка основных параметров
u0 = [990, 10, 0]
tmax = 60.0  # Мы увеличиваем время, чтобы увидеть конец медленных эпидемий
betas = [0.03, 0.05, 0.07, 0.09]

sensitivity_results = DataFrame(
    beta = Float64[],
    peak_I = Int64[],
    peak_time = Float64[],
    total_infected = Int64[]
)

p_all = plot(title="Анализ чувствительности : изменение β", xlab="Время", ylab="Количество инфицированных (I)")

println("Начало анализа чувствительности")

# перебирайте различные значения бета-версии
for β in betas
    Random.seed!(1234)
    p = [β, 10.0, 0.25] # β variable, c=10, γ=0.25
    m = MakeSIRModel(u0, p)
    activate(m)
    sir_run(m, tmax)
    data = out(m)
    max_I = maximum(data.I)                # Пик заражения
    t_peak = data.t[argmax(data.I)]        # Время, когда наступает пик
    final_R = last(data.R)                 # Общее количество людей, у которых было заболевание

    push!(sensitivity_results, (β, max_I, t_peak, final_R))
    plot!(p_all, data.t, data.I, label="β = $β", lw=2)
    println("Завершено для β = $β : Пик из $max_I до t=$t_peak")
end
```

Рис. 5: Рисунок 5

- Отображение на консоли. (рис. 6).

```
Завершено для β = 0.03 : Пик из 36 до t=20.122628096582936
Завершено для β = 0.05 : Пик из 306 до t=10.889577010531282
Завершено для β = 0.07 : Пик из 575 до t=8.52085205966972
Завершено для β = 0.09 : Пик из 666 до t=7.232872771986825

Краткое изложение результатов :
Could not create decoration from factory! Running with no decorations.
4×4 DataFrame
 Row │ beta     peak_I  peak_time  total_infected
     │ Float64  Int64   Float64    Int64
─────┼──────────────────────────────────────────
   1 │    0.03      36   20.1226              280
   2 │    0.05     306   10.8896              761
   3 │    0.07     575    8.52085             922
   4 │    0.09     666    7.23287             968
```

Рис. 6: Рисунок 6

- График, который мы получили. (рис. 7).

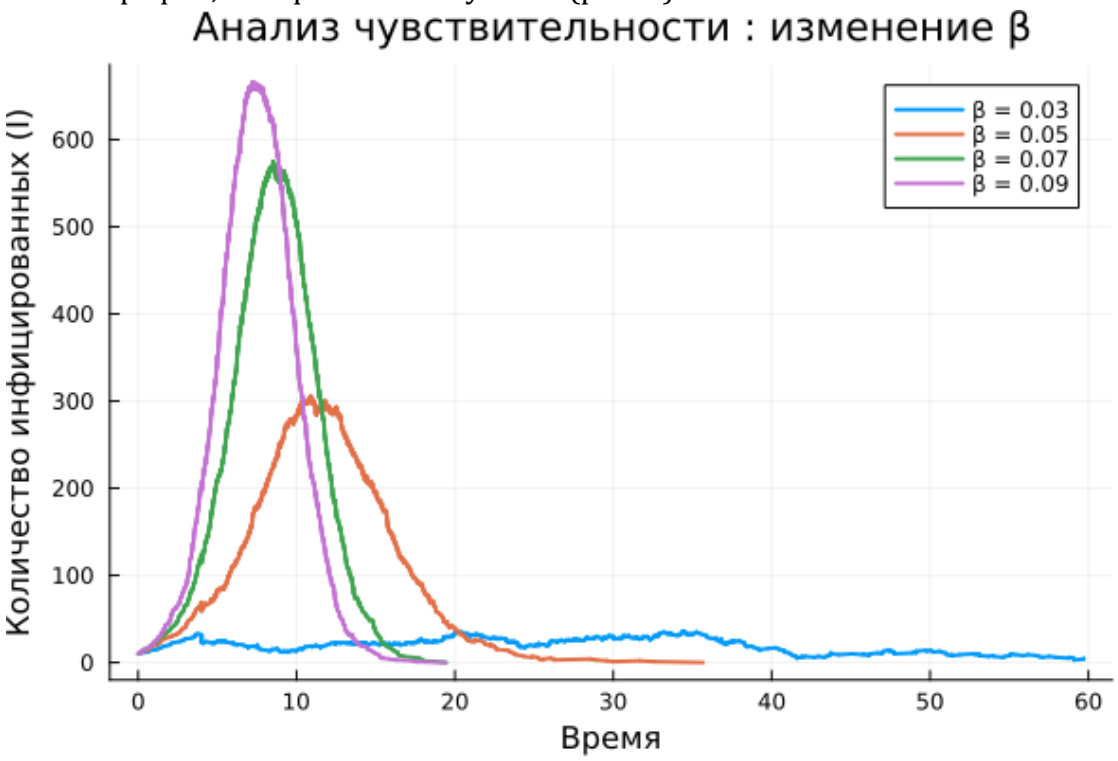


Рис. 7: Рисунок 7

Детерминированная длительность болезни

- В src/sir_model.jl в функции live заменить строку

```
@yield timeout(env, rand(Exponential(1/m.$/gamma$)))
```

на

```
@yield timeout(env, 1/m.$/gamma$)
```

(рис. 8).

```
@resumable function live(env::ConcurrentSim.Simulation, individual::SIRPerson, m::SIRModel)
    while individual.status == :S
        @yield timeout(env, rand(Exponential(1/m.c)))
        alter = individual
        while alter == individual
            N = length(m.allIndividuals)
            index = rand(DiscreteUniform(1, N))
            alter = m.allIndividuals[index]
        end
        if alter.status == :I
            if rand(Uniform(0, 1)) < m.β
                individual.status = :I
                infection_update!(env, m)
            end
        end
    end
    if individual.status == :I
        @yield timeout(env, 1/m.γ)
        individual.status = :R
        recovery_update!(env, m)
    end
end
```

Рис. 8: Рисунок 8

Оценка производительности

Добавить в скрипт:

```
using BenchmarkTools
@benchmark sir_run(des_model, tmax)
```

(рис. 9).

```
Запуск benchmark для N = 10 000
BenchmarkTools.Trial: 1 sample with 1 evaluation per sample.
 Single result which took 5.507 s (5.59% GC) to evaluate,
 with a memory estimate of 857.30 MiB, over 34312275 allocations.
```

Рис. 9: Рисунок 9

Вакцинация

- Мы добавим в модель срабатывание события вакцинации по таймеру (рис. 10).

```
@resumable function vaccination_campaign(env::ConcurrentSim.Simulation, m::SIRModel, coverage::Float64,
t_start::Float64)
    @yield timeout(env, t_start)
    compteur_vaccines = 0
    for ind in m.allIndividuals
        if ind.status == :S && rand() < coverage
            ind.status = :R
            compteur_vaccines += 1
        end
    end
    if compteur_vaccines > 0
        push!(m.ta, ConcurrentSim.now(env))
        push!(m.Sa, m.Sa[end] - compteur_vaccines)
        push!(m.Ia, m.Ia[end])
        push!(m.Ra, m.Ra[end] + compteur_vaccines)

        println("Campagne de vaccination à t=$t_start : $compteur_vaccines personnes vaccinées.")
    end
end
```

Рис. 10: Рисунок 10

Модель SEIR

- Ввести латентный период (статус :E). После заражения индивид переходит в :E и через время, распределённое экспоненциально с параметром */sigma*, становится инфекционным (:I). Оценить, как меняется динамика по сравнению с SIR.

- Отображение на консоли. (рис. 11).

```
--- Démarrage de la simulation SEIR avec Démographie ---
Simulation en cours pour tf = 300.0...
Simulation terminée.
Population finale : 9928
```

Рис. 11: Рисунок 11

- График, который мы получили. (рис. 12).

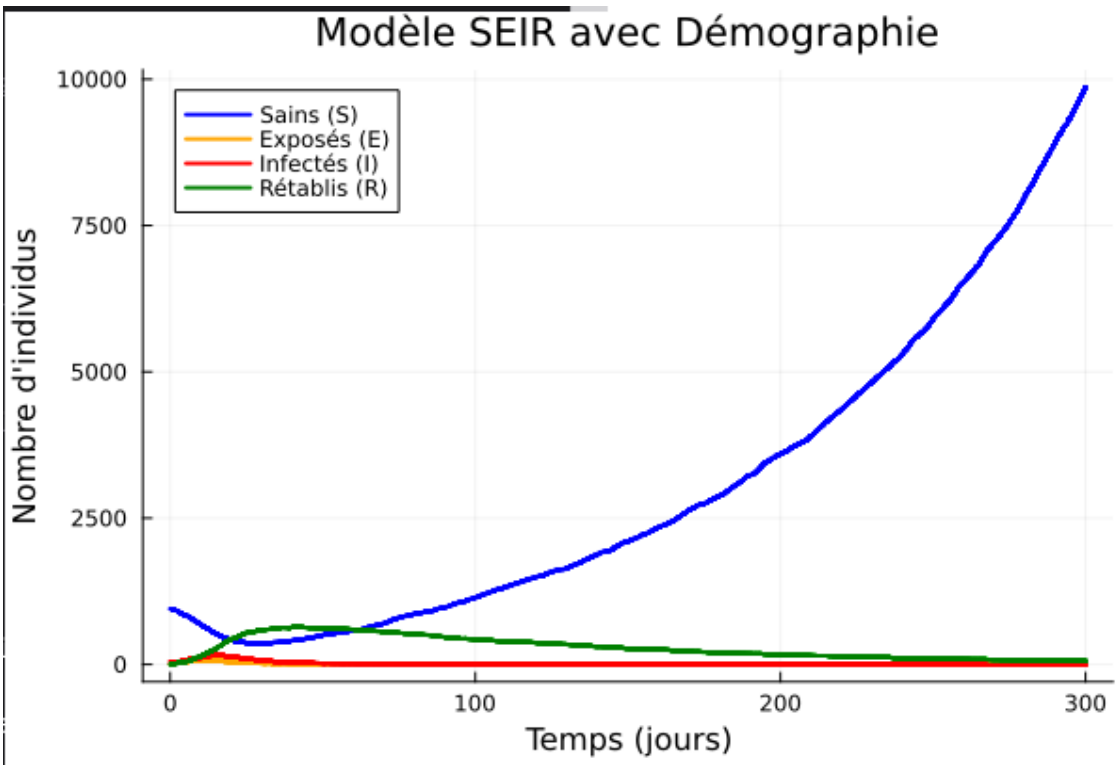


Рис. 12: Рисунок 12

5 Выводы

- В результате выполнения лабораторной работы была Реализация основных моделей в дискретно-событийном подходе.

Список литературы